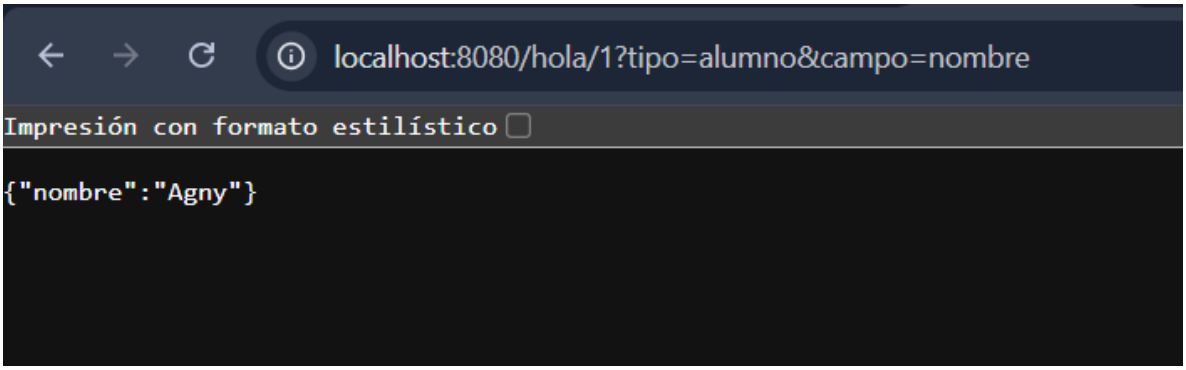
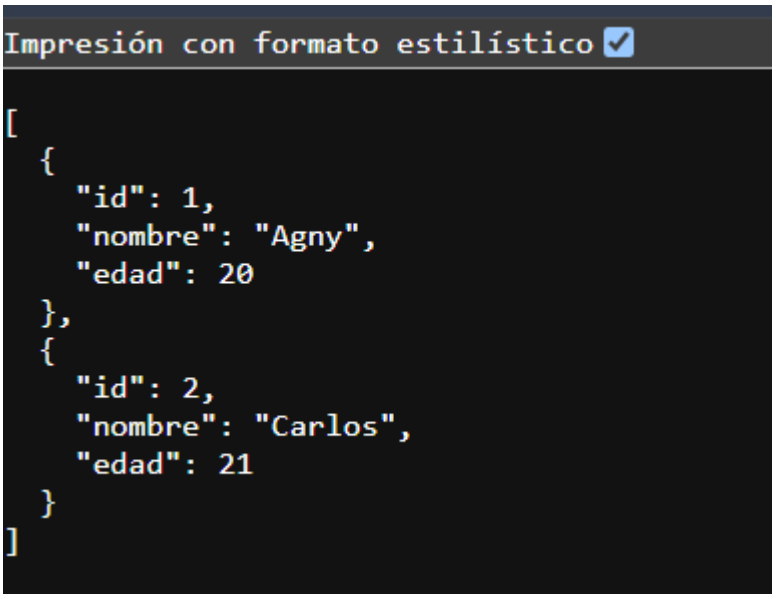
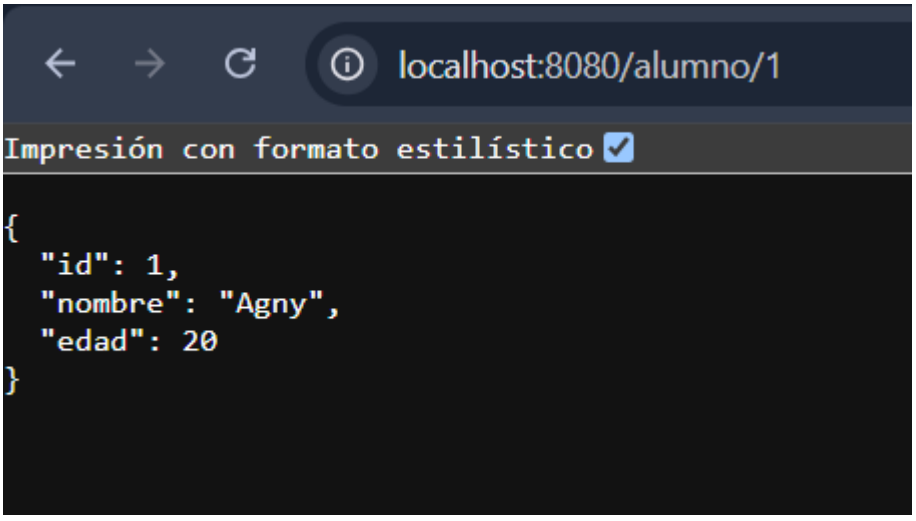
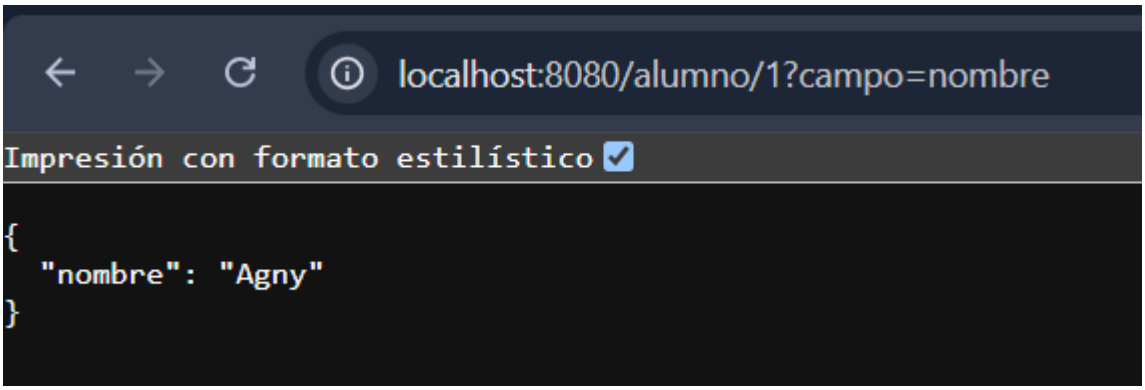
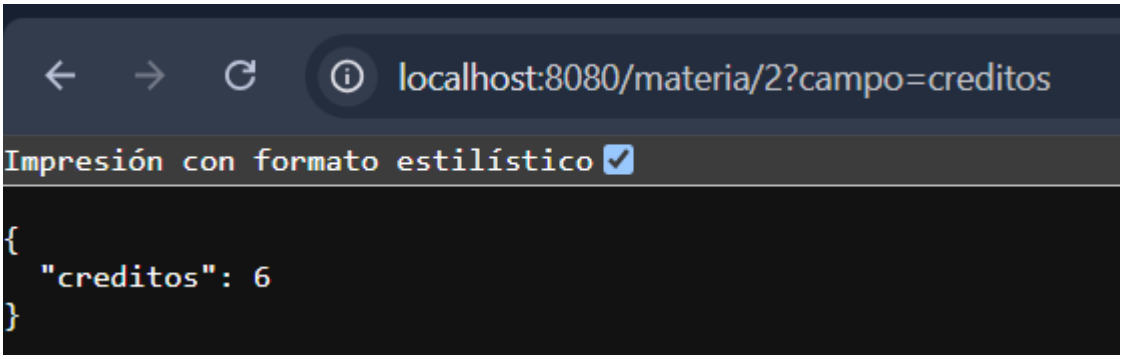

localhost:8080/hola
Impresión con formato estilístico
[
{
"id": 1,
"alumno": {
"nombre": "Agny",
"edad": 20
},
"materia": {
"nombre": "Programación",
"creditos": 8
},
"escuela": {
"nombre": "UTTEC",
"ciudad": "México"
}
},
{
"id": 2,
"alumno": {
"nombre": "Carlos",
"edad": 21
},
"materia": {
"nombre": "Matemáticas",
"creditos": 6
},
"escuela": {
"nombre": "UTTEC",
"ciudad": "México"
}
}
]

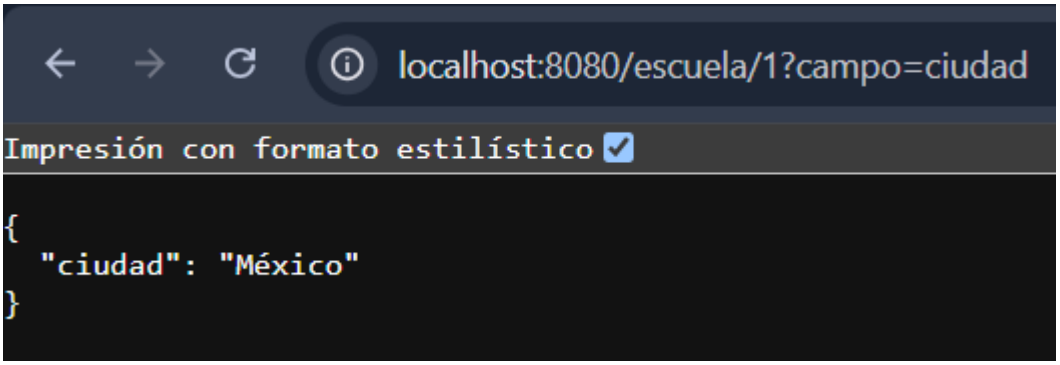

localhost:8080/hola/1?tipo=alumno&campo=nombre
Impresión con formato estilístico
{"nombre":"Agny"}

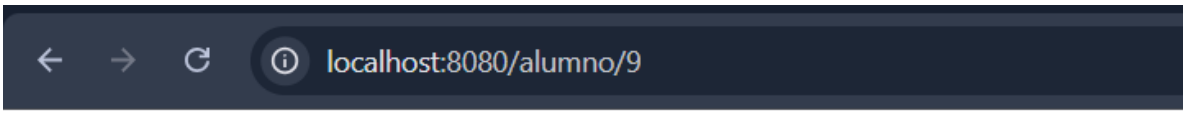

Impresión con formato estilístico
[
{
"id": 1,
"nombre": "Agny",
"edad": 20
},
{
"id": 2,
"nombre": "Carlos",
"edad": 21
}
]


localhost:8080/alumno/1
Impresión con formato estilístico
{
"id": 1,
"nombre": "Agny",
"edad": 20
}





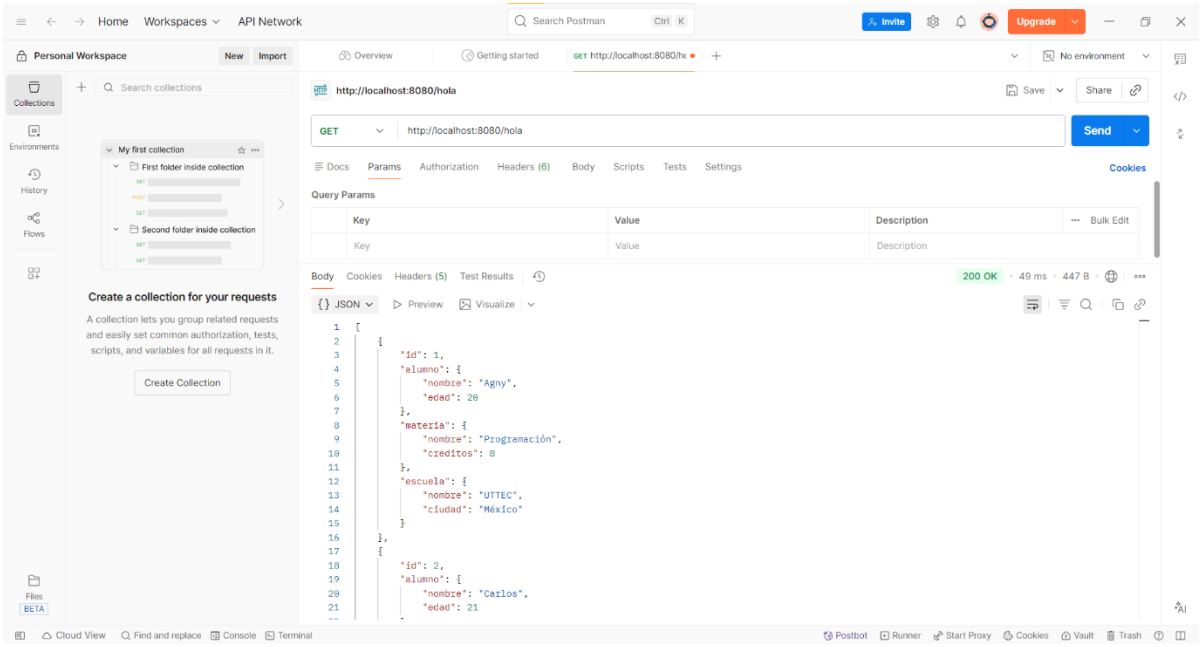


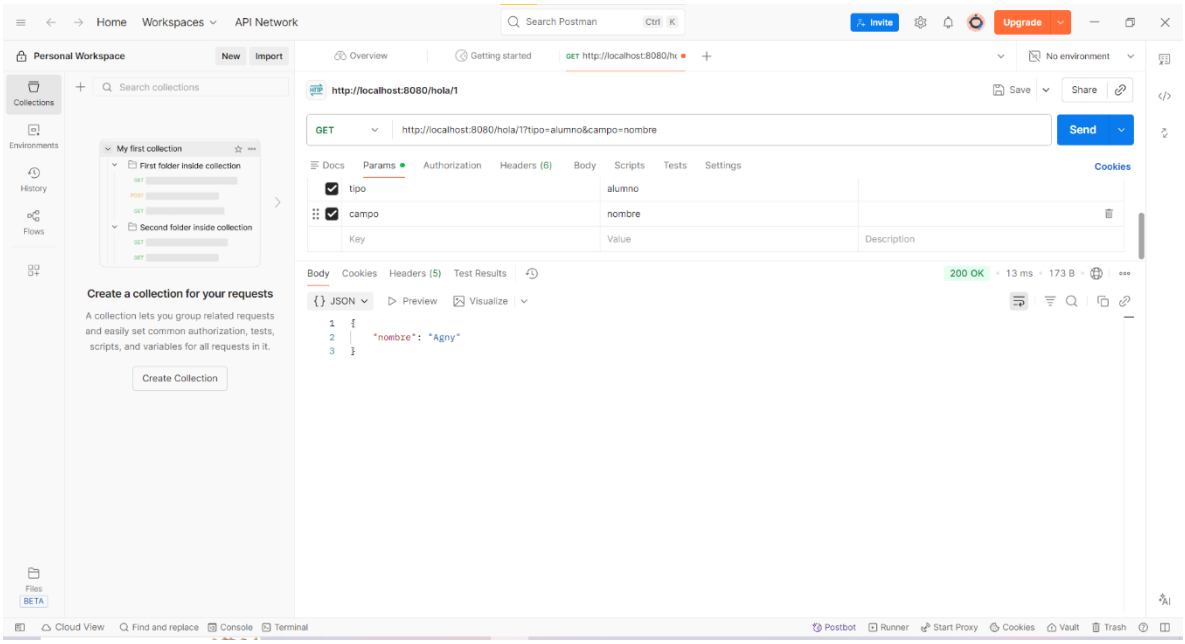


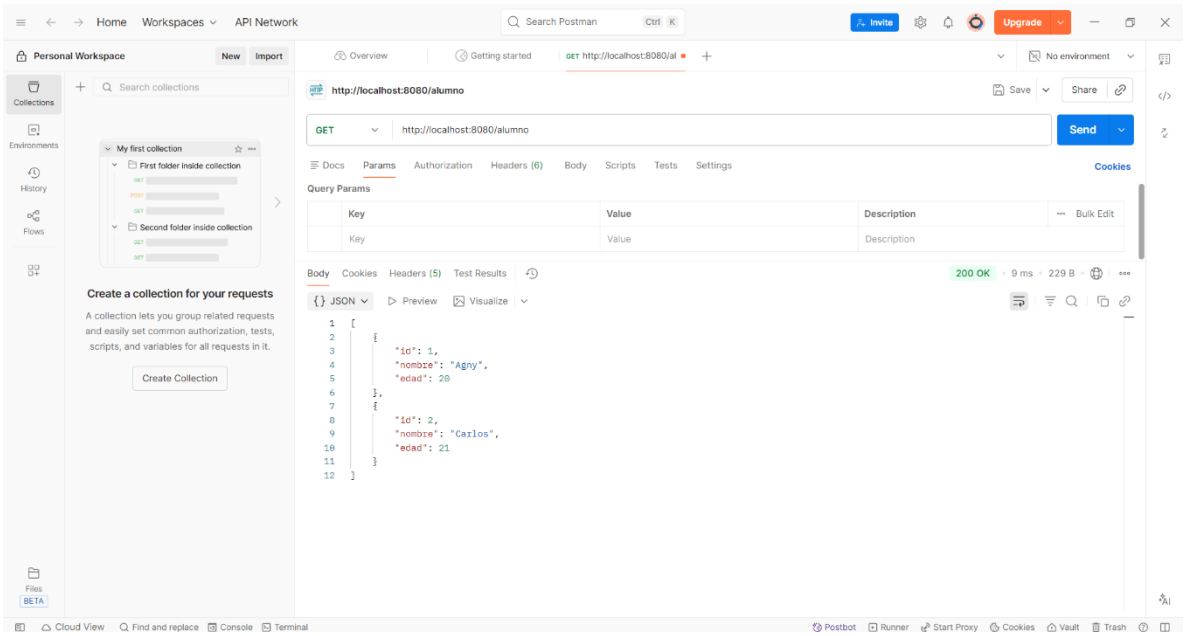
Whitelabel Error Page

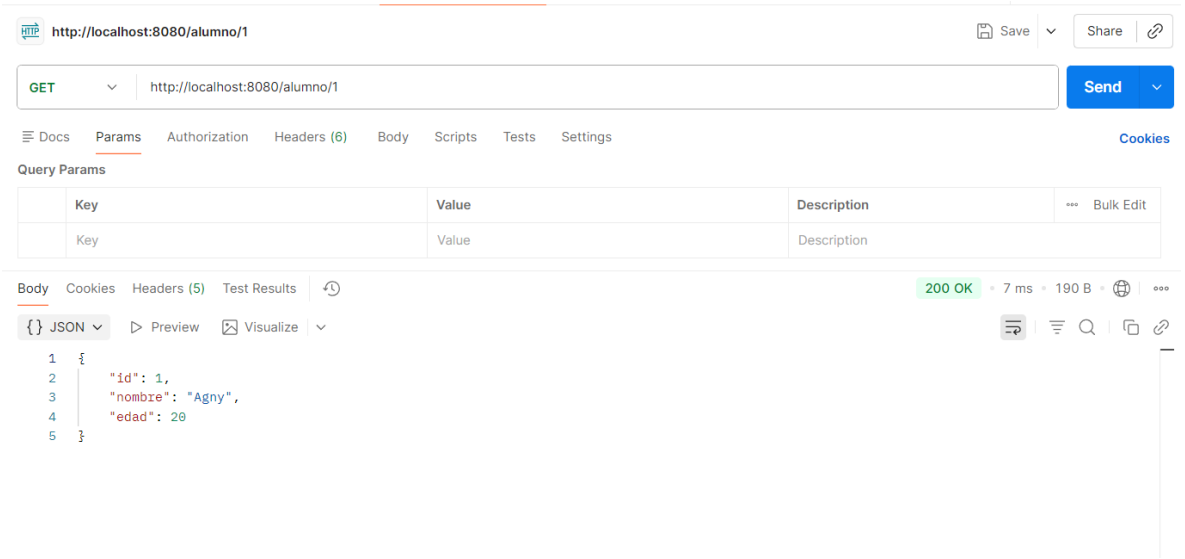
This application has no explicit mapping for /error, so you are seeing this as a fallback.

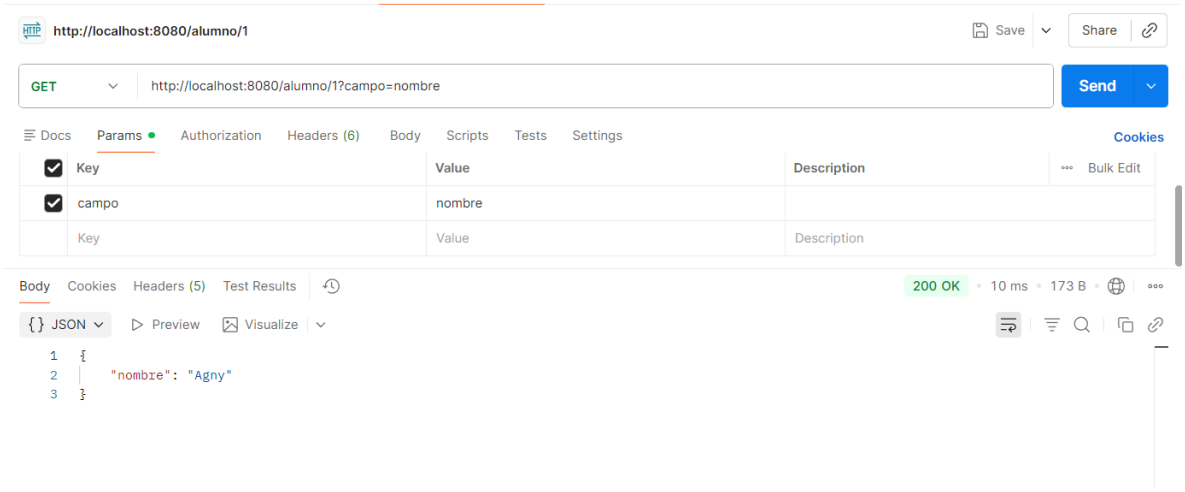
Mon Oct 12 23:56:34 CST 2026
There was an unexpected error (type=Not Found, status=404).

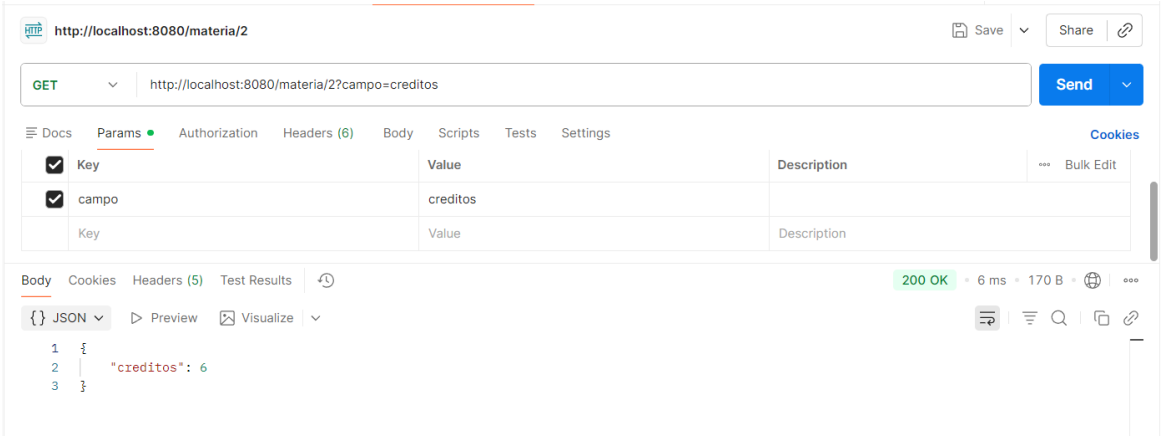

Home
Workspaces
API Network
Search Postman
Ctrl K
Invite
Upgrade
Personal Workspace
New
Import
Overview
Getting started
GET http://localhost:8080/hc
No environment
Collections
Environments
History
Flows
Files
BETA
Search collections
My first collection
First folder inside collection
Second folder inside collection
Create a collection for your requests
A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.
Create Collection
http://localhost:8080/hola
Save
Share
GET
http://localhost:8080/hola
Send
Docs
Params
Authorization
Headers (6)
Body
Scripts
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body
Cookies
Headers (5)
Test Results
200 OK
49 ms
447 B
JSON
Preview
Visualize
[
{
"id": 1,
"alumno": {
"nombre": "Agny",
"edad": 20
},
"materia": {
"nombre": "Programación",
"creditos": 8
},
"escuela": {
"nombre": "UTTEC",
"ciudad": "México"
}
},
{
"id": 2,
"alumno": {
"nombre": "Carlos",
"edad": 21
Cloud View
Find and replace
Console
Terminal
Postbot
Runner
Start Proxy
Cookies
Vault
Trash

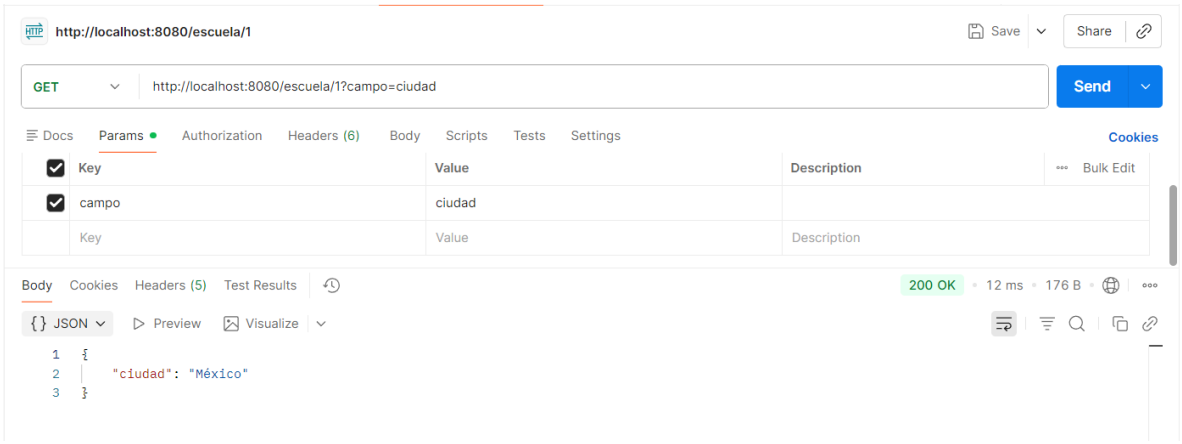

Home
Workspaces
API Network
Search Postman
Ctrl K
Invite
Upgrade
Personal Workspace
New
Import
Overview
Getting started
GET http://localhost:8080/hc
No environment
Collections
Environments
History
Flows
Files
BETA
Search collections
My first collection
First folder inside collection
Second folder inside collection
Create a collection for your requests
A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.
Create Collection
http://localhost:8080/hola/1
Save
Share
GET
http://localhost:8080/hola/1?tipo=alumno&campo=nombre
Send
Docs
Params
Authorization
Headers (6)
Body
Scripts
Tests
Settings
Cookies
tipo
alumno
campo
nombre
Key
Value
Description
Body
Cookies
Headers (5)
Test Results
200 OK
13 ms
173 B
JSON
Preview
Visualize
{
"nombre": "Agny"
}
Cloud View
Find and replace
Console
Terminal
Postbot
Runner
Start Proxy
Cookies
Vault
Trash

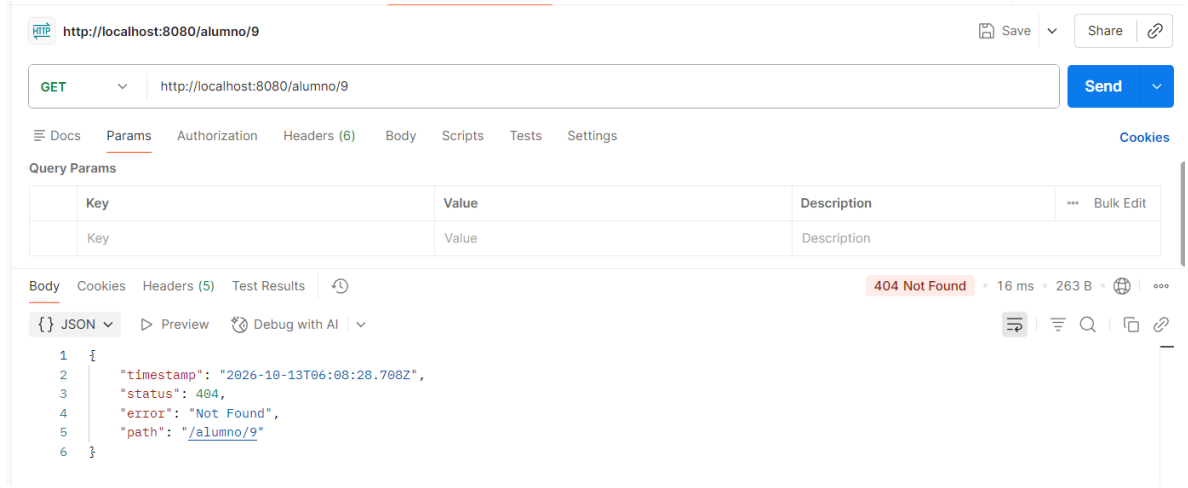

Home
Workspaces
API Network
Search Postman
Invite
Upgrade
Personal Workspace
New
Import
Collections
Environments
History
Flows
Files
BETA
Search collections
My first collection
First folder inside collection
Second folder inside collection
Create a collection for your requests
A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.
Create Collection
Overview
Getting started
GET http://localhost:8080/al
No environment
http://localhost:8080/alumno
Save
Share
GET
http://localhost:8080/alumno
Send
Docs
Params
Authorization
Headers (6)
Body
Scripts
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body
Cookies
Headers (5)
Test Results
200 OK
9 ms
229 B
JSON
Preview
Visualize
[
{
"id": 1,
"nombre": "Agny",
"edad": 20
},
{
"id": 2,
"nombre": "Carlos",
"edad": 21
}
]
Cloud View
Find and replace
Console
Terminal
Postbot
Runner
Start Proxy
Cookies
Vault
Trash


http://localhost:8080/alumno/1
Save
Share
GET
http://localhost:8080/alumno/1
Send
Docs
Params
Authorization
Headers (6)
Body
Scripts
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body
Cookies
Headers (5)
Test Results
200 OK
7 ms
190 B
JSON
Preview
Visualize
{
"id": 1,
"nombre": "Agny",
"edad": 20
}


http://localhost:8080/alumno/1
Save
Share
GET
http://localhost:8080/alumno/1?campo=nombre
Send
Docs
Params
Authorization
Headers (6)
Body
Scripts
Tests
Settings
Cookies
Key
Value
Description
Bulk Edit
campo
nombre
Key
Value
Description
Body
Cookies
Headers (5)
Test Results
200 OK
10 ms
173 B
{ } JSON
Preview
Visualize
1 {
2 "nombre": "Agny"
3 }


http://localhost:8080/materia/2
Save
Share
GET
http://localhost:8080/materia/2?campo=creditos
Send
Docs
Params
Authorization
Headers (6)
Body
Scripts
Tests
Settings
Cookies
Key
Value
Description
Bulk Edit
campo
creditos
Key
Value
Description
Body
Cookies
Headers (5)
Test Results
200 OK
6 ms
170 B
{ } JSON
Preview
Visualize
1 {
2 "creditos": 6
3 }


http://localhost:8080/escuela/1
Save
Share
GET
http://localhost:8080/escuela/1?campo=ciudad
Send
Docs
Params
Authorization
Headers (6)
Body
Scripts
Tests
Settings
Cookies
Key
Value
Description
Bulk Edit
campo
ciudad
Key
Value
Description
Body
Cookies
Headers (5)
Test Results
200 OK
12 ms
176 B
JSON
Preview
Visualize
{
"ciudad": "México"
}


http://localhost:8080/alumno/9
Save
Share
GET
http://localhost:8080/alumno/9
Send
Docs
Params
Authorization
Headers (6)
Body
Scripts
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers (5)
Test Results
404 Not Found
16 ms
263 B
JSON
Preview
Debug with AI
{
"timestamp": "2026-10-13T06:08:28.708Z",
"status": 404,
"error": "Not Found",
"path": "/alumno/9"
}

SaludoController

```
package com.example.demo.controller;


import java.util.List;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;


@RestController
public class SaludoController {


  private final List<Datos> lista = List.of(
      new Datos(1,
          new Alumno("Agny", 20),
          new Materia("Programación", 8),
          new Escuela("UTTEC", "México")),
      new Datos(2,
          new Alumno("Carlos", 21),
          new Materia("Matemáticas", 6),
          new Escuela("UTTEC", "México"))
  );
```

```
@GetMapping("/hola")
public List<Datos> todos() {
   return lista;
}

@GetMapping("/hola/{id}")
public Object porId(@PathVariable("id") int id,
                    @RequestParam(value = "tipo", required = false) String tipo,
                    @RequestParam(value = "campo", required = false) String campo) {

   Datos d = buscar(id);

   if (tipo == null || tipo.isBlank()) {
      return d;
   }

   boolean sinCampo = (campo == null || campo.isBlank());

   switch (tipo) {
      case "alumno":
         if (sinCampo) return d.alumno();
         if (campo.equals("nombre")) return Map.of("nombre", d.alumno().nombre());
         if (campo.equals("edad")) return Map.of("edad", d.alumno().edad());
         break;
      case "materia":
         if (sinCampo) return d.materia();
```

```
            if (campo.equals("nombre")) return Map.of("nombre", d.materia().nombre());
            if (campo.equals("creditos")) return Map.of("creditos", d.materia().creditos());
            break;
        case "escuela":
            if (sinCampo) return d.escuela();
            if (campo.equals("nombre")) return Map.of("nombre", d.escuela().nombre());
            if (campo.equals("ciudad")) return Map.of("ciudad", d.escuela().ciudad());
            break;
        default:
            throw new ResponseStatusException(HttpStatus.NOT_FOUND, "No existe el
tipo " + tipo);
      }
      throw new ResponseStatusException(HttpStatus.NOT_FOUND, "No existe el
campo " + campo);
   }

   private Datos buscar(int id) {
      return lista.stream()
            .filter(d -> d.id() == id)
            .findFirst()
            .orElseThrow(() -> new ResponseStatusException(
                  HttpStatus.NOT_FOUND, "No existe el ID " + id));
   }

   public record Datos(int id, Alumno alumno, Materia materia, Escuela escuela) {}
   public record Alumno(String nombre, int edad) {}
   public record Materia(String nombre, int creditos) {}
```

```
    public record Escuela(String nombre, String ciudad) {}
}
```

EscuelaController

```
package com.example.demo.controller;

import java.util.List;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;

@RestController
public class EscuelaController {

    public record EscuelaId(int id, String nombre, String ciudad) {}

    private final List<EscuelaId> lista = List.of(
            new EscuelaId(1, "UTTEC", "México"),
            new EscuelaId(2, "UNAM", "México")
    );

    // Todas: /escuela
```

```
  @GetMapping("/escuela")
  public List<EscuelaId> todas() {
    return lista;
  }

  // Por ID: /escuela/1  o  /escuela/1?campo=ciudad
  @GetMapping("/escuela/{id}")
  public Object porId(@PathVariable("id") int id,
                      @RequestParam(value = "campo", required = false) String campo) {

    EscuelaId e = lista.stream()
        .filter(x -> x.id() == id)
        .findFirst()
        .orElseThrow(() -> new ResponseStatusException(
            HttpStatus.NOT_FOUND, "No existe la escuela con ID " + id));

    if (campo == null || campo.isBlank()) return e;
    if (campo.equals("nombre")) return Map.of("nombre", e.nombre());
    if (campo.equals("ciudad")) return Map.of("ciudad", e.ciudad());

    throw new ResponseStatusException(HttpStatus.NOT_FOUND, "No existe el
campo " + campo);
  }
}
```

AlumnoController

```
package com.example.demo.controller;


import java.util.List;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;


@RestController
public class AlumnoController {

  public record AlumnoId(int id, String nombre, int edad) {}


  private final List<AlumnoId> lista = List.of(
        new AlumnoId(1, "Agny", 20),
        new AlumnoId(2, "Carlos", 21)
  );


  // Todos: /alumno
  @GetMapping("/alumno")
  public List<AlumnoId> todos() {
    return lista;
```

```
  }

  // Por ID: /alumno/1  o  /alumno/1?campo=nombre
  @GetMapping("/alumno/{id}")
  public Object porId(@PathVariable("id") int id,
            @RequestParam(value = "campo", required = false) String campo) {

    AlumnoId a = lista.stream()
        .filter(x -> x.id() == id)
        .findFirst()
        .orElseThrow(() -> new ResponseStatusException(
            HttpStatus.NOT_FOUND, "No existe el alumno con ID " + id));

    if (campo == null || campo.isBlank()) return a;
    if (campo.equals("nombre")) return Map.of("nombre", a.nombre());
    if (campo.equals("edad")) return Map.of("edad", a.edad());

    throw new ResponseStatusException(HttpStatus.NOT_FOUND, "No existe el
campo " + campo);
  }
}
```

MateriaController

```
package com.example.demo.controller;

import java.util.List;
```

```
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;

@RestController
public class MateriaController {

  public record MateriaId(int id, String nombre, int creditos) {}

  private final List<MateriaId> lista = List.of(
      new MateriaId(1, "Programación", 8),
      new MateriaId(2, "Matemáticas", 6)
  );

  // Todas: /materia
  @GetMapping("/materia")
  public List<MateriaId> todas() {
    return lista;
  }

  // Por ID: /materia/1  o  /materia/1?campo=creditos
  @GetMapping("/materia/{id}")
```

```
  public Object porId(@PathVariable("id") int id,
                @RequestParam(value = "campo", required = false) String campo) {


    MateriaId m = lista.stream()
        .filter(x -> x.id() == id)
        .findFirst()
        .orElseThrow(() -> new ResponseStatusException(
            HttpStatus.NOT_FOUND, "No existe la materia con ID " + id));


    if (campo == null || campo.isBlank()) return m;
    if (campo.equals("nombre")) return Map.of("nombre", m.nombre());
    if (campo.equals("creditos")) return Map.of("creditos", m.creditos());


    throw new ResponseStatusException(HttpStatus.NOT_FOUND, "No existe el
campo " + campo);
  }
}
```

DemoApplication

```
package com.example.demo;


import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;


@SpringBootApplication
public class DemoApplication {
   public static void main(String[] args) {
      SpringApplication.run(DemoApplication.class, args);
   }
}
```